

I want you to help me design and plan a complete freeware chat application with the following specifications:

PROJECT OVERVIEW

Create a lightweight, cross-platform text messaging application called "BLABBERBOX" with a humorous, playful theme. The app focuses purely on encrypted text-based messaging between users with zero bloatware or unnecessary features.

APPLICATION NAME & BRANDING

- **App Name:** BLABBERBOX
- **Tagline:** "Where gossip meets encryption—chat away, worry-free!"
- **Theme:** Playful, sarcastic, and minimalist
- **Mascot Concept:** A quirky cartoon chattering mouth or gossip owl
- **Color Scheme:** Vibrant (neon pink, electric blue, lime green) with dark mode support
- **Tone:** Funny, irreverent, self-aware humor in all UI copy

FUNNY KEYWORDS & UI LANGUAGE

Instead of standard messaging terms, use these:

- "Send" → "Blurt it out" or "Drop the truth bomb"
- "Messages" → "Blabber Feed"
- "Online" → "Actively Yapping"
- "Typing..." → "Cooking up nonsense..."
- "User" → "Blabber"
- "Chat" → "Gossip Session"
- "New Message" → "Fresh Tea 🍵"
- "Settings" → "Silence the Haters"

- "Blocked" → "Muted the Fool"
- "Last seen" → "Last caught yapping at"
- "Delete Message" → "Unsay that tragedy"
- "Clear Chat" → "Forget this ever happened"
- Error Messages: "Oops! Even our servers have bad days."

CORE FEATURES (TEXT-ONLY)

What to Include:

1. User Authentication (secure registration/login)
2. Text Messaging (single and group chats)
3. Emoji & Text Formatting Support
4. Typing Indicators
5. Read Receipts (optional toggle)
6. Online/Offline Status
7. User Profiles (username, bio, avatar)
8. Block/Unblock Users
9. Message Search
10. Chat Backup & Export (encrypted)
11. Dark Mode
12. Message Reactions (emoji responses)
13. Message Editing & Deletion
14. Timestamps
15. Chat Threading/Replies (optional)

Explicitly Exclude:

- Voice messages

- Video calls
- Audio calls
- Phone number authentication
- Video sharing
- Live location sharing
- Stories/Status features

SECURITY & ENCRYPTION REQUIREMENTS

- **Protocol:** End-to-End Encryption (E2EE) using TLS 1.3 for transport
- **Algorithm:** Signal Protocol / Double Ratchet Algorithm (industry standard)
- **Key Exchange:** Perfect Forward Secrecy (PFS)
- **Backend:** Zero-knowledge architecture (server cannot read messages)
- **Data Storage:**
 - Messages encrypted at rest on server
 - Local encrypted database on client
 - No plain text data stored anywhere
- **Privacy:** No tracking, no ads, no data selling
- **Compliance:** GDPR compliant, open-source security auditable

TECHNICAL REQUIREMENTS

Backend Architecture:

- Technology: Node.js/Express, Python/Django, or Go
- Database: PostgreSQL (encrypted fields)
- Message Queue: Redis (for real-time messaging)
- Hosting: Self-hosted or cloud (AWS, Digital Ocean) with E2EE

- API: RESTful with WebSocket support for real-time chat

Frontend (Web/Desktop):

- Framework: React, Vue, or Electron + React
- Language: TypeScript
- State Management: Redux or Zustand
- Real-time: Socket.io or WebSocket
- Encryption Library: crypto-js, TweetNaCl.js, or libsodium.js

Mobile (iOS & Android):

- Framework: React Native, Flutter, or Native
- Database: SQLite (encrypted with SQLCipher)
- Real-time: Firebase Cloud Messaging or custom WebSocket
- Security: Use platform-native encryption APIs
- Store: Google Play Store & Apple App Store (free)

PLATFORM & DEPLOYMENT REQUIREMENTS

- ☒ Desktop: Windows, macOS, Linux (Electron App)
- ☒ Mobile: iOS 12+, Android 8+
- ☒ Web: Progressive Web App (PWA) optional
- ☒ All platforms must have feature parity
- ☒ 100% Freeware (no premium features, no ads, no paywalls)
- ☒ Open Source (GitHub public repository)
- ☒ Auto-updates enabled for security patches

UI/UX DESIGN PRINCIPLES

1. **Minimalist**: Clean, distraction-free interface
2. **Consistent**: Same design across all platforms
3. **Fast**: Sub-100ms message delivery target
4. **Accessible**: WCAG 2.1 AA compliant
5. **Funny**: Witty placeholder text, humorous empty states
 - Empty chat: "No gossip yet. We're disappointed too."
 - No internet: "Internet's ghosting us."
 - Load error: "Our hamsters are tired. Refresh?"
6. **Dark-friendly**: Dark mode default option

DEVELOPMENT ROADMAP

Phase 1 (MVP):

- User auth system
- 1-to-1 messaging
- Basic encryption
- Desktop + mobile builds

Phase 2:

- Group chats
- Message search
- User profiles
- Message reactions

Phase 3:

- Message threading

- Chat backup
- Advanced settings
- Security audit

****Phase 4:****

- Performance optimization
- Additional languages
- Community features

SUCCESS METRICS

- Message encryption working end-to-end
- Cross-platform compatibility
- <100ms message delivery
- 99.9% uptime
- Zero data breaches
- Active open-source community

ADDITIONAL NOTES

- Code should be well-documented and open-source friendly
- Include API documentation (Swagger/OpenAPI)
- Provide setup guides for self-hosting
- Include security audit results
- Add rate limiting to prevent spam
- Implement user-friendly onboarding
- Ensure CAPTCHA for registration
- Add two-factor authentication (2FA) option

Please provide:

1. Complete architecture diagram
2. Database schema design
3. API endpoint specifications
4. Security implementation details
5. Step-by-step development guide
6. Deployment instructions
7. Code structure recommendations
8. Testing strategy
9. Timeline estimate
10. Potential challenges & solutions